

Security at Scale

Phase 3 · Advanced · Estimated time: 2 hours

1. Concept From First Principles

Every chapter so far has assumed requests come from legitimate, well-behaved clients. This chapter covers the layer that actually enforces that assumption: **authentication** (confirming who someone is) and **authorization** (confirming what they're allowed to do), plus the encryption and common attack vectors every backend engineer needs working knowledge of before an HLD interview, since security is a near-guaranteed follow-up topic once a design involves user data or payments.

Authentication vs. authorization: a distinction interviewers check for

Authentication (AuthN) answers "who are you?" — verifying an identity, typically via a password, an OTP, or a token. **Authorization (AuthZ)** answers "what are you allowed to do?" — checking permissions *after* identity is established. These are genuinely separate concerns that a surprising number of engineers conflate: a request can be perfectly authenticated (we know exactly who this user is) and still be unauthorized (this user isn't allowed to view someone else's order details). Getting this distinction crisp is table stakes for any security-adjacent interview answer.

Session-based auth vs. token-based auth

Session-based authentication is the traditional model: on login, the server creates a session record (server-side state) and gives the client a session ID (usually in a cookie); every subsequent request includes that session ID, and the server looks it up to know who's making the request. This is simple and lets the server instantly revoke a session (just delete the server-side record), but it requires shared, centralized session storage across a horizontally-scaled fleet of app servers (Phase 1 Chapter 9's statelessness principle would otherwise be broken) — typically solved with a shared Redis-backed session store.

Token-based authentication, most commonly using **JWT (JSON Web Tokens)**, takes a different approach: on login, the server issues a signed token containing the user's identity and claims (roles, permissions) directly inside the token itself. Any server receiving a request with that token can verify its signature and trust its contents *without* looking anything up in shared storage, because the token is self-contained and cryptographically signed — a genuinely stateless approach that fits naturally with horizontally-scaled microservices (Phase 2 Chapter 11), since no service needs to share session state with any other to validate a request.

Backend engineer analogy

A session ID is like a coat-check ticket: meaningless on its own, useless without the coat-check counter (server-side session store) to look it up against. A JWT is like a sealed, signed, tamper-evident envelope containing your ID card inside it: anyone with the right key can verify the seal and read the contents directly, without calling back to whoever issued it. This is exactly why JWTs are popular in microservices — each service can verify a request independently, without a shared session lookup.

JWT's statelessness comes with a genuine, often-overlooked cost: **revocation is hard**. Because a valid, signed JWT is trusted purely based on its signature and expiry time, there's no simple way to instantly invalidate one specific token before it naturally expires (unlike a session, which can be deleted from the store immediately) — the standard mitigations are keeping JWT expiry times short (minutes, not days) combined with a separate, longer-lived **refresh token** that the client exchanges for a new short-lived JWT periodically, and maintaining a small denylist of explicitly revoked tokens for the rare cases (like a detected account compromise) where immediate revocation genuinely matters.

OAuth 2.0: delegated authorization

OAuth 2.0 solves a different, related problem: letting a user grant a third-party application limited access to their data on another service, without ever sharing their actual password with that third party ("let this app post to your Instagram" without giving the app your Instagram password). OAuth is fundamentally an *authorization* framework, not an authentication one, though it's frequently used as the foundation for "Sign in with Google/Facebook"-style authentication flows (via the related OpenID Connect layer built on top of OAuth). JWTs and OAuth solve genuinely different problems and are commonly used together, not competing alternatives — a common interview confusion worth resolving explicitly.

Encryption and common attack vectors, briefly

Encryption in transit (TLS/HTTPS, Phase 1 Chapter 4) protects data moving across the network; **encryption at rest** protects data sitting in a database or object store (Phase 1 Chapter 11), typically handled transparently by the database or cloud provider, with sensitive fields (payment details, government IDs) sometimes given an additional layer of field-level encryption. On common attack vectors, an HLD interview generally wants awareness rather than deep exploitation detail: **SQL injection** (mitigated by parameterized queries, never string-concatenated SQL), **rate limiting** (Phase 2 Chapter 10) as a defense against brute-force and denial-of-service attempts, and the principle of **least privilege** (every service, credential, and user should have only the minimum access it genuinely needs, limiting the damage any single compromised credential can do).

2. Real-World Example / Case Study

Razorpay's and PhonePe's merchant-facing APIs use token-based authentication (API keys and, for user-facing flows, JWTs) precisely because their infrastructure is horizontally scaled across many stateless services — a session-store lookup on every single payment API call would add both latency and a shared-infrastructure dependency that a self-contained, signed token avoids entirely.

Swiggy's and Talabat's mobile apps typically use short-lived JWT access tokens paired with longer-lived refresh tokens, exactly the pattern this chapter describes, letting the app silently refresh the

user's session in the background without requiring re-login every few minutes, while keeping the actual exposure window of a stolen access token small.

A Careem or Noon integration allowing users to "Sign in with Google" is built on OAuth 2.0/OpenID Connect — the user grants the app permission to read their basic Google profile without ever sharing their Google password with Careem or Noon directly, a clean, real-world illustration of OAuth's delegated-authorization model in a consumer product context.

3. Diagram

Session-based auth (stateful):	Token-based auth / JWT (stateless):
Login -> Server creates session Client gets: session_id (cookie) Every request: session_id --> Server looks up session_id in SHARED session store (Redis) Easy to revoke: delete the row	Login -> Server issues signed JWT Client gets: JWT (contains claims) Every request: JWT in header --> Server VERIFIES signature only NO shared lookup needed Hard to revoke: token is valid until it expires (mitigate with short expiry + refresh tokens)
OAuth 2.0 (delegated authorization):	
User -> "Allow App X to access your Google profile?" -> Google Google issues App X a limited-scope ACCESS TOKEN App X never sees the user's actual Google password	

4. Trade-offs Table

Aspect	Session-based auth	Token-based auth (JWT)
State	Server-side (needs shared store)	Stateless (self-contained token)
Revocation	Instant — delete the session record	Hard — needs short expiry + refresh tokens or denylist
Microservices fit	Needs shared session store across services	Any service can verify independently
Payload size	Small (just an ID)	Larger (contains claims, sent on every request)
Best fit	Simpler monoliths, easy revocation needs	Distributed/microservices systems (Phase 2, Ch 11)

5. Common Interview Questions

Q1: "Explain the difference between authentication and authorization with an example."

Model approach: Define AuthN as verifying identity and AuthZ as verifying permissions, then give a concrete example where both apply separately — a logged-in user (authenticated) trying to view another user's private order details (unauthorized, despite being authenticated).

Q2: "Why is revoking a JWT harder than revoking a session, and how do you mitigate that?"

Model approach: Explain that a JWT's validity is based purely on its signature and expiry, not a server-side lookup, so there's no single place to instantly invalidate one; mitigate with short-lived access tokens plus refresh tokens, and a denylist for rare, urgent revocation cases.

Q3: "Would you use session-based or token-based auth for a microservices architecture, and why?"

Model approach: Recommend token-based (JWT), justified by each service being able to independently verify a request without a shared session-store dependency, directly connecting to Phase 2 Chapter 11's microservices trade-offs.

Q4: "What is OAuth 2.0, and how is it different from JWT?"

Model approach: Explain OAuth as a delegated-authorization framework (letting a third party access your data without sharing your password) versus JWT as a token *format* that's often used to implement the access tokens OAuth issues — the two are complementary, not competing.

6. Practice Exercises

- 1 Design the authentication flow for a mobile app using short-lived JWT access tokens and longer-lived refresh tokens, including what happens when the access token expires mid-session.
- 2 Explain why a session-based auth system needs a shared Redis-backed session store once you scale beyond one app server, connecting this to Phase 1 Chapter 9's statelessness principle.
- 3 Design an authorization check for an order-tracking API that must ensure a user can only view their own orders, not any order ID they happen to guess.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"JWT Explained In Under 10 Minutes (JSON Web Tokens)"	Clear, focused explanation of JWT structure and mechanics.
English	"API Authentication EXPLAINED! OAuth vs JWT vs API Keys"	Directly clarifies the OAuth-vs-JWT distinction this chapter emphasizes.
Hindi	"[HINDI] Introduction to JWT JSON Web Token Security Concerns of JWT"	Hindi introduction to JWT covering both mechanics and security concerns.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers authentication and authorization as part of a full bilingual course.

8. Key Takeaways

- Authentication verifies identity ('who are you'); authorization verifies permissions ('what can you do').
- Session-based auth is server-stateful, easy to revoke, but needs shared session storage across a scaled fleet.
- Token-based auth (JWT) is self-contained and stateless, fitting microservices well, but hard to revoke instantly.

- Short-lived access tokens plus longer-lived refresh tokens are the standard mitigation for JWT's revocation weakness.
- OAuth 2.0 is a delegated-authorization framework, distinct from but often paired with JWT as the token format.
- Encryption in transit (TLS) and at rest protect data at different stages; both are typically required.
- Parameterized queries, rate limiting, and least-privilege access are baseline defenses every HLD design should mention.